《Ai Agent》第3-17节：增加调度器策略执行Agent链路

来自： 码农会锁

小傅哥 2025年09月06日 19:35

本章重点 ： ★★★☆☆
课程视频 ： https://t.zsxq.com/44qnU
代码分支 ： https://gitcode.net/KnowledgePlanet/ai-agent-station-study/-/tree/3-17-agent-dispatch-strategy
工程代码 ： https://gitcode.net/KnowledgePlanet/ai-agent-station-study

版权说明 ： ©本项目与星球签约合作，受《中华人民共和国著作权法实施条例》版权法保护，禁止任何理由和任何方式公开(public)源码、资料、视频等小傅哥发布的星球内容到Github、Gitee等各类平台，违反可追究进一步的法律责任。

作者：小傅哥
博客：https://bugstack.cn

> 沉淀、分享、成长，让自己和他人都能有所收获！😄

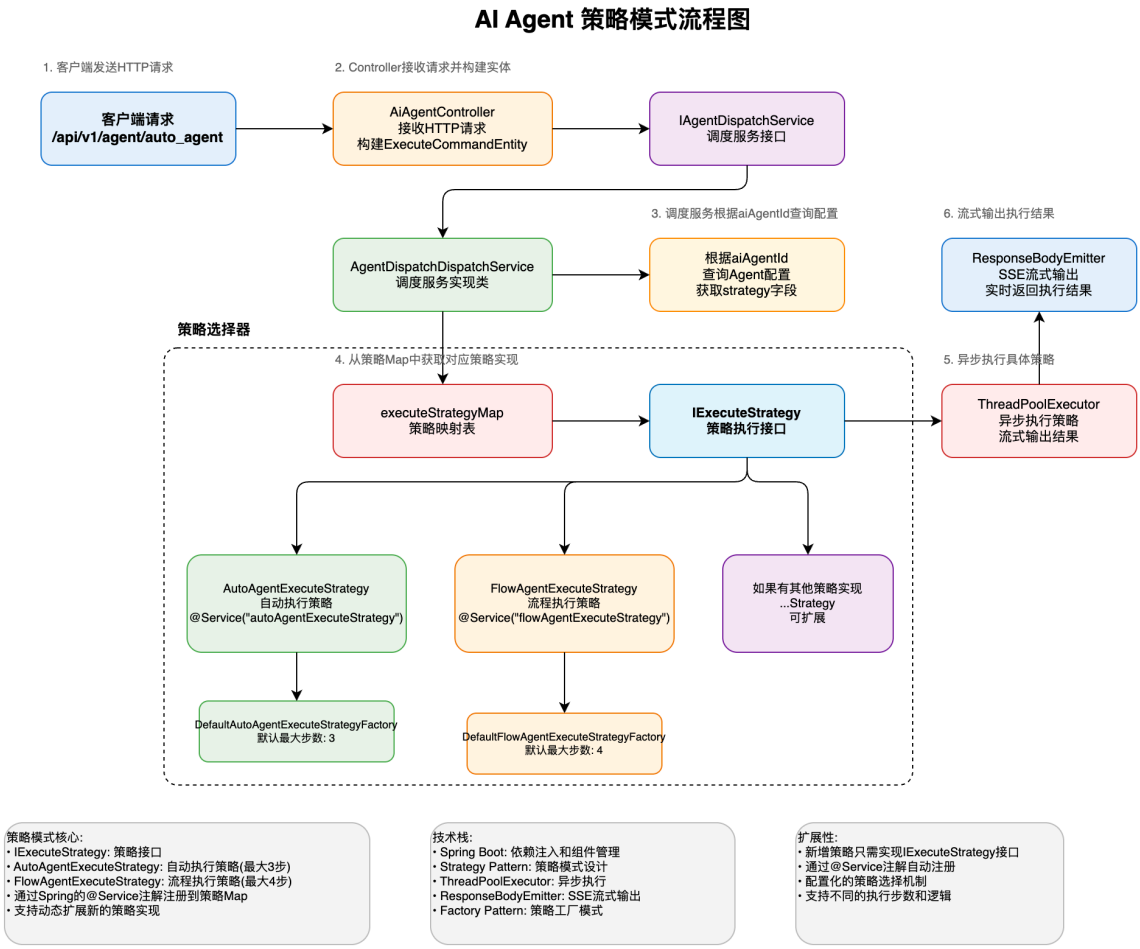
一、本章诉求
二、流程设计
三、编码实现
1. 工程结构
2. 修改说明
3. 库表数据
4. 策略调度器
5. 接口层处理
6. 完成结果通知
四、测试验证
五、读者作业：

一、本章诉求

从ui页面开始，增加 ai agent 分类选择使用。因为这里有不同类型的 ai agent，所以要对调用的过程增加一个策略调度器，按照不同类型的 ai agent 选择不同的执行策略。

二、流程设计

如图，Ai Agent 策略调度器执行过程；



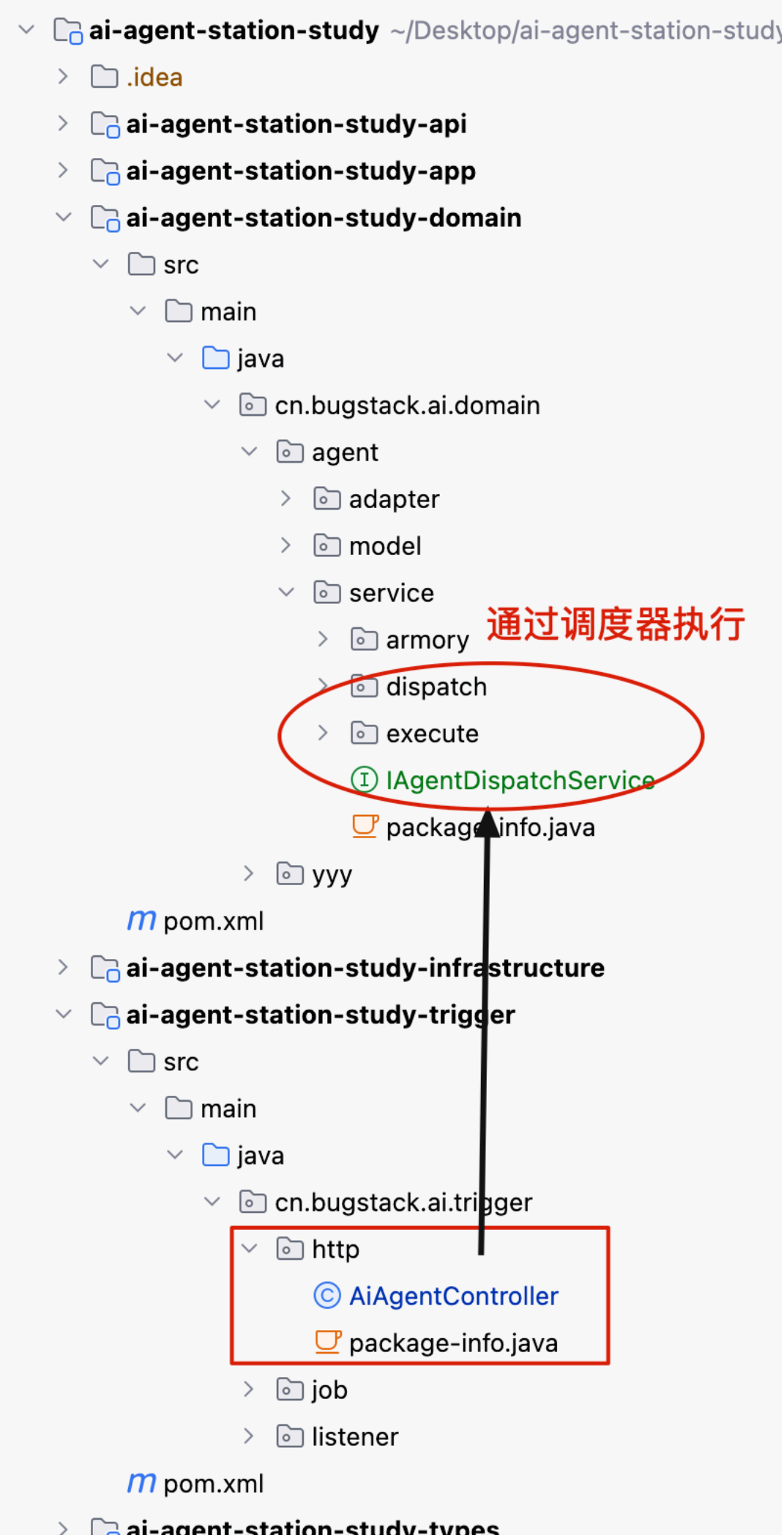
首先，我们实现的 ai agent 有多种类型，所以在选择场景选择的时候，要根据不同的场景获取到走那种类型的 ai agent 执行策略。

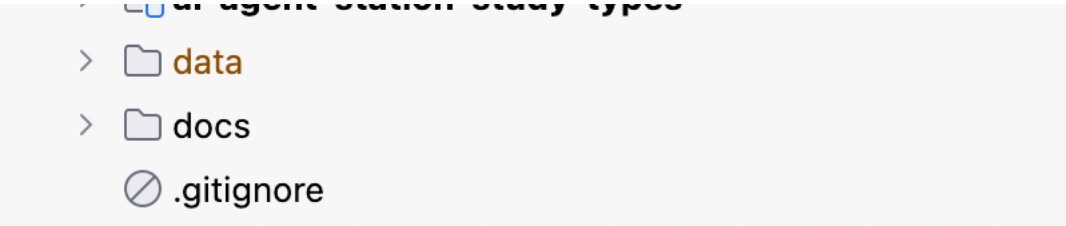
之后，这里我们就把 AutoAgent、FlowAgent 都放到调度器里执行，另外要在数据库表 ai_agent 中增加一个 strategy 执行策略配置，这样用户提问时候传入的 agent id 就可以获取到对应的策略了。

最后，本节还有一点关于页面 UI 的完善，每次对话，会把对话消息存储到历史对话中。这部分前端的东西使用 ai 开发工具处理的。

三、编码实现

1. 工程结构


ai-agent-station-study ~/Desktop/ai-agent-station-study
.idea
ai-agent-station-study-api
ai-agent-station-study-app
ai-agent-station-study-domain
src
main
java
cn.bugstack.ai.domain
agent
adapter
model
service
armory
通过调度器执行
dispatch
execute
IAgentDispatchService
package-info.java
yyy
pom.xml
ai-agent-station-study-infrastructure
ai-agent-station-study-trigger
src
main
java
cn.bugstack.ai.trigger
http
AiAgentController
package-info.java
job
listener
pom.xml
ai-agent-station-study-types



AiAgentController，执行用户请求时，之前的方式是注入了对应的 auoAgent 执行策略。那么这里需要增加一个调度器服务接口，通过调度器，选择 execute 下的 autoAgent 和 flowAgent

因为需要判断是哪个类型的 Agent 所以需要库表（ai_agent）增加字段，这部分库表增加后，对应的代码里也增加处理。

2. 修改说明

增加 aiAgentDao.queryByAgentId(aiAgentId) 接口，返回数据库表 agent 执行策略配置。

增加 IAgentDispatchService 调度器接口，去调度执行策略。执行策略通过 private Map<String, IExecuteStrategy> executeStrategyMap; 注入。

修改 Step4ExecuteStepsNode、FlowAgentExecuteStrategy，增加完成标识，就是最终结果通过 sse 返回给前端。

修改 AiAgentController 调用 agent 策略方式，通过调度器执行。

前端页面部分的优化是使用 ai 完成的，ai 对于前端的编码很好处理。（后端人员不需要过度关注前端，面试这部分不会问的）

3. 库表数据

库表 ai_agent 增加 strategy 策略字段。

4. 策略调度器

```
/**
 * Agent 策略调度器接口
 * @author xiaofuge bugstack.cn @小傅哥
 * 2025/9/2 06:54
 */
public interface IAgentDispatchService {

    void dispatch(ExecuteCommandEntity requestParameter, ResponseBodyEmitter emitter) throws Excep

}

@Slf4j
@Service
public class AgentDispatchDispatchService implements IAgentDispatchService {

    @Resource
    private Map<String, IExecuteStrategy> executeStrategyMap;

    @Resource
    private IAgentRepository repository;

    @Resource
    private ThreadPoolExecutor threadPoolExecutor;

    @Override
    public void dispatch(ExecuteCommandEntity requestParameter, ResponseBodyEmitter emitter) throw
        AiAgentVO aiAgentVO = repository.queryAiAgentByAgentId(requestParameter.getAiAgentId());
```

```
        String strategy = aiAgentVO.getStrategy();
        IExecuteStrategy executeStrategy = executeStrategyMap.get(strategy);
        if (null == executeStrategy) {
            throw new BizException("不存在的执行策略类型 strategy:" + strategy);
        }

        // 3. 异步执行AutoAgent
        threadPoolExecutor.execute(() -> {
            try {
                executeStrategy.execute(requestParameter, emitter);
            } catch (Exception e) {
                log.error("AutoAgent执行异常: {}", e.getMessage(), e);
                try {
                    emitter.send("执行异常: " + e.getMessage());
                } catch (Exception ex) {
                    log.error("发送异常信息失败: {}", ex.getMessage(), ex);
                }
            } finally {
                try {
                    emitter.complete();
                } catch (Exception e) {
                    log.error("完成流式输出失败: {}", e.getMessage(), e);
                }
            }
        });

    }

}
```

这部分也简单，通过策略调度器，来执行调用哪个策略。对应是什么策略，则可以根据库表数据获取

5. 接口层处理

```
@Slf4j
@RestController
@RequestMapping("/api/v1/agent")
@CrossOrigin(origins = "*", allowedHeaders = "*", methods = {RequestMethod.GET, RequestMethod.POST
public class AiAgentController implements IAiAgentService {

    @Resource(name = "autoAgentExecuteStrategy")
    private IExecuteStrategy autoAgentExecuteStrategy;

    @Resource
    private ThreadPoolExecutor threadPoolExecutor;

    @Resource
    private IAgentDispatchService agentDispatchService;

    @RequestMapping(value = "auto_agent", method = RequestMethod.POST)
    public ResponseBodyEmitter autoAgent(@RequestBody AutoAgentRequestDTO request, HttpServletResp
        log.info("AutoAgent流式执行请求开始，请求信息: {}", JSON.toJSONString(request));

        try {
            // 设置SSE响应头
            response.setContentType("text/event-stream");
            response.setCharacterEncoding("UTF-8");
            response.setHeader("Cache-Control", "no-cache");
            response.setHeader("Connection", "keep-alive");

            // 1. 创建流式输出对象
```

```
            ResponseBodyEmitter emitter = new ResponseBodyEmitter(Long.MAX_VALUE);

            // 2. 构建执行命令实体
            ExecuteCommandEntity executeCommandEntity = ExecuteCommandEntity.builder()
                    .aiAgentId(request.getAiAgentId())
                    .message(request.getMessage())
                    .sessionId(request.getSessionId())
                    .maxStep(request.getMaxStep())
                    .build();

            // 3. 调度处理
            agentDispatchService.dispatch(executeCommandEntity, emitter);

            return emitter;

        } catch (Exception e) {
            log.error("AutoAgent请求处理异常: {}", e.getMessage(), e);
            ResponseBodyEmitter errorEmitter = new ResponseBodyEmitter();
            try {
                errorEmitter.send("请求处理异常: " + e.getMessage());
                errorEmitter.complete();
            } catch (Exception ex) {
                log.error("发送错误信息失败: {}", ex.getMessage(), ex);
            }
            return errorEmitter;
        }
    }

}
```

原本接口层需要根据注入的策略进行调度，但因为多种类型，不好区分。所以这里可以使用调度器来处理。

6. 完成结果通知

```
    /**
     * 发送总结结果到流式输出
     */
    private void sendSummaryResult(DefaultFlowAgentExecuteStrategyFactory.DynamicContext dynamicCo
        // 构建执行总结内容
        StringBuilder summaryContent = new StringBuilder();
        summaryContent.append("## 执行步骤完成总结\n\n");

        // 获取执行历史
        StringBuilder executionHistory = dynamicContext.getExecutionHistory();
        if (executionHistory != null && executionHistory.length() > 0) {
            summaryContent.append("### 已完成的工作\n");
            summaryContent.append(executionHistory.toString());
            summaryContent.append("\n\n");
        }

        summaryContent.append("### 执行状态\n");
        summaryContent.append("✅ 所有规划步骤已成功执行完成\n\n");

        summaryContent.append("### 执行效果评估\n");
        summaryContent.append("📊 任务执行流程顺利完成，各步骤按计划执行");

        AutoAgentExecuteResultEntity result = AutoAgentExecuteResultEntity.createSummaryResult(
                summaryContent.toString(), sessionId);
        sendSseResult(dynamicContext, result);
        log.info("📊 已发送总结结果到【最终执行结果】区域");
    }
```

```
/**
 * 发送完成标识到流式输出
 */
private void sendCompleteResult(DefaultFlowAgentExecuteStrategyFactory.DynamicContext dynamicC
    AutoAgentExecuteResultEntity result = AutoAgentExecuteResultEntity.createCompleteResult(se
    sendSseResult(dynamicContext, result);
    log.info("✅ 已发送完成标识");
}
```

在 Step4ExecuteStepsNode、FlowAgentExecuteStrategy 两个地方要添加完成结果通知。

四、测试验证

1. 在测试时候要把最新的库表数据导入到你的数据库里，之后执行启动服务访问ui界面。
2. yml 配置文件，注意加载 2101,2102,2103，以便于测试本节场景。也可以加载你前面学习的其他的，一起验证。

进入页面后，选择第一个CSDN发帖+通知。之后提问案例里有对应的内容，可以选择进行提问。

五、读者作业：

简单作业：因为本节代码很少，也很清晰。你可以对照课程所讲内容，和对应的代码说明，来编写实现。

复杂作业：尝试增加一种新的 mcp 服务的场景，让自己有更多的思考，也让简历有独属于你的东西。

知识星球